

Analyze the two GenAI programs. The analysis must include:

- a. Name of two GenAI programs\
 - i. Chat GPT and Gemni
- b. How you accessed them (e.g., Google query, CoPilot on my computer)
 - i. I used a google browser to access Chat GPT and Gemni
- c. Prompt used. (You must use the same prompt for both GenAIs.)
 - i. "I'm trying to make a C program of a number guessing game. Picking a number 1-10 and the user only has three tries. After each attempt have the user see how many tries they have left and it tells them if its too high or low"
 - ii. Although I had to add an extra prompt to gemni to make the code without extra libraries.
- d. Generated code from both GenAIs.
 - i. <https://chatgpt.com/c/6a99fdf6-7060-83ea-9617-2894167a4a8e>

```
1.
2. #include <stdio.h>
3.
4. int main() {
5.
6.     int secretNumber = 7;
7.     int guess;
8.     int tries = 3;
9.
10.    while (tries > 0) {
11.
12.        printf("Guess a number between 1 and 10: ");
13.        scanf("%d", &guess);
14.
15.        if (guess == secretNumber) {
16.            printf("Correct! You guessed the number!\n");
17.            break;
18.        }
19.        else if (guess > secretNumber) {
20.            printf("Too high!\n");
21.        }
22.        else {
23.            printf("Too low!\n");
24.        }
```

```

25.
26.     tries--;
27.
28.     printf("You have %d tries left.\n\n", tries);
29. }
30.
31. if (tries == 0 && guess != secretNumber) {
32.     printf("You are out of tries! The number was
    %d.\n", secretNumber);
33. }
34.
35.     return 0;
36. }

```

ii. <https://gemini.google.com/app/cf2a3d5d72cffdf6>

```

1. #include <stdio.h>
2.
3. int main() {
4.     int secret = 7; // The number to guess (1-10)
5.     int guess;
6.     int tries = 3;
7.
8.     printf("Guess the secret number between 1 and
    10!\n");
9.
10.    for (int i = 0; i < tries; i++) {
11.        int remaining = tries - i;
12.
13.        printf("Enter your guess: ");
14.        scanf("%d", &guess);
15.
16.        if (guess == secret) {
17.            printf("You got it right!\n");
18.            return 0; // Stop the program early if they
    win
19.        }
20.

```

```

21.         if (guess < secret) {
22.             printf("Too low! ");
23.         } else {
24.             printf("Too high! ");
25.         }
26.
27.         printf("Tries left: %d\n\n", remaining - 1);
28.     }
29.
30.     printf("Game over! The secret number was %d.\n",
        secret);
31.     return 0;
32. }

```

- e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.
 - i. Correctness: When analyzing the correctness of the code I found that they both run well. But in both they don't have error cases if the user accidentally inputs a string instead.
 - ii. Execution Time: In execution time Gemni uses a for-loop and Chat GPT uses a while loop. The for-loop is the better loop to make sure the code is as fast as possible.
 - iii. Space Complexity: They both have similar set ups but the for-loop in Gemni's code uses less memory in comparison to the while loop in Chat GPT it uses more memory.
 - iv. Maintainability: Chat GPT has no comments and Gemni has little to none but it uses some to tell where specific code is.
- f. Select one to use as the basis for this assignment and justify why you picked it.
 - i. I would pick Gemni's code since it is more organized in comparison to Chat GPT. It is the most efficient in following my prompt and it runs faster and better than Chat GPT.
- g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.
 - i. Correctness: I would have an error case just in case the user inputs a string. Even though we assume the user wouldn't put in a string but just to make sure it's alright.
 - ii. Execution Time: I wouldn't need to change the execution time since it already uses a for loop and only has three guesses.
 - iii. Space Complexity: The space complexity wouldn't need any changing since it only has a few integers. The memory used doesn't change it is at $O(1)$
 - iv. Maintainability: Add comments to each line of code and add a small description to make sure you know what the program does.

3. Make the changes identified in 2g above.
4. Compile and debug your program as described in the “C Basics” lecture.
5. Push both your source code, final cycle server executable file, and PDF of your GenAI analysis to your GitHub repository.